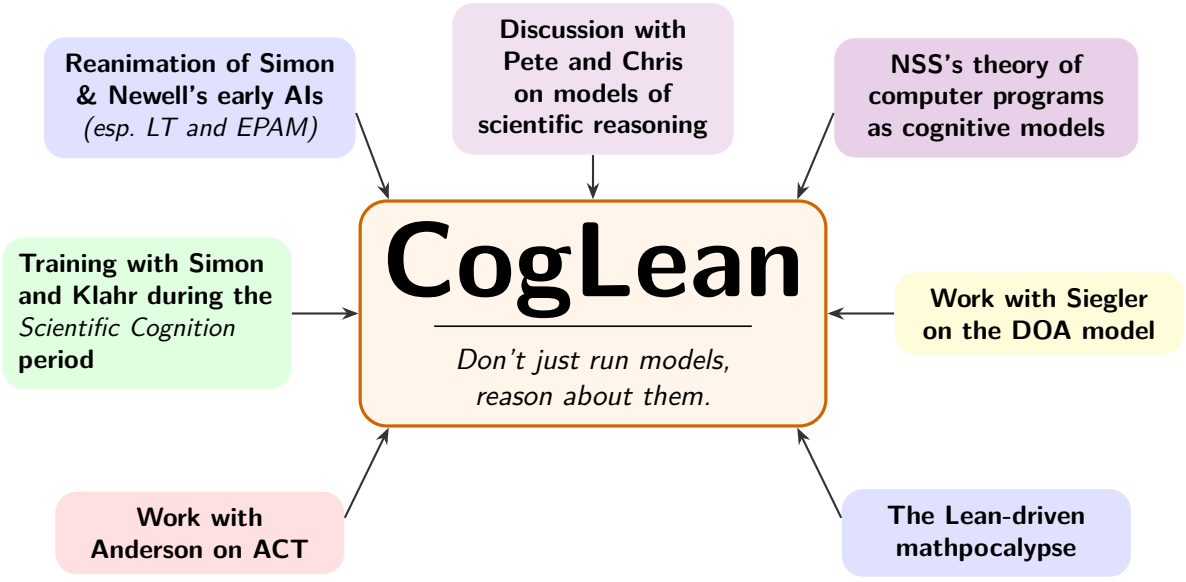
Epoofany


Reanimation of Simon & Newell's early AIs (esp. LT and EPAM)
Discussion with Pete and Chris on models of scientific reasoning
NSS's theory of computer programs as cognitive models
Training with Simon and Klahr during the Scientific Cognition period
CogLean
Don't just run models, reason about them.
Work with Siegler on the DOA model
Work with Anderson on ACT
The Lean-driven mathpocalypse

CogLean

Towards a Cumulative, Machine-Checked Theory of Cognition

Jeff Shrager

November 2026

Draft for Comment

The problem: making theory cumulative

- Newell (1973): twenty questions with nature, answers never assembled; Newell (1990): unified theories.
- Cognitive architectures (Soar, ACT-R; now the Common Model) have spent decades on this: **shared mechanisms**, so models of different tasks reuse and integrate theoretical commitments.
- What they accumulate is mainly **models**: what a model predicts is established by running it, one setting at a time, and a result for one model is rarely reusable *as a result* by another.
- CogLean, **complementary**: accumulate machine-checked **theorems** about mechanisms, reusable deductively, and stated across architectural commitments.

Can cognitive theory be organized the way Mathlib organizes mathematics: definitions made once, results proved, and later work building on earlier results?

Formalizing explanations

- **Phenomena**: regularities of behavior, stated without any mechanism. Observed, they are **psychological facts**; derived but not yet observed, they are **predictions**.
- **Mechanisms and assumptions**: postulates, conditions, laws.
- **Explanations**: deductions of a phenomenon from mechanisms and assumptions (a theorem and its lemmas).
- **Deduction**: what Lean checks. Given these mechanisms and assumptions, the phenomenon follows; whether they hold of people is for experiments.

The explanation is a proof about **the same program that runs**: change the model, and every explanation it breaks fails to check.

A phenomenon can have several explanations; a mechanism can help explain several phenomena.

Prior formal approaches to cognitive theory

Work	Formalism	What it establishes
Soar specified in Z (Milnes et al. 1992)	Z notation	a complete, low-level, non-executable specification of one architecture
Sceptic (Cooper, Fox, Farringdon & Shallice 1996)	executable specification language	models (Soar among them) stated in a language designed for cognitive models
Reconstructing ACT-R (Stewart & West 2007)	Python reimplementation	core commitments separated from implementation; alternative components
ACT-R operational semantics (Gall & Frühwirth 2018)	abstract semantics; CHR	a translation proved sound and complete; confluence tests
ACT-R as timed automata (Langenfeld et al. 2019, 2021)	timed automata	exhaustive defect analysis of models; the semantics proved correct
A common formal ground for architectures (Ragni et al. 2018)	formal framework	ACT-R and Soar embedded for comparison
Soar to PRISM (Ganeriwala et al. 2026)	Markov chains, PRISM	probabilistic model checking of stochastic models
Verbal theories as logic programs; as automata (Rohner & Kjellerstrand 2021; Finkel et al. 2025)	Prolog; automata	what a verbal theory's premises entail
Critical tests from core assumptions (Fisher et al. 2025)	proofs in prose over model classes	which processing structures ACT-R's core rules out

The analogy: Mathlib

- Mathlib, the community library of formalized mathematics in Lean, proposes no new mathematics.
- It is a **shared environment**: definitions made once, every result machine-checked, later work citing earlier results.
- A theorem about groups, once proved, is available to everyone who uses groups.

CogLean asks whether cognitive theory can be organized the same way.
It is not a new theory of cognition, nor a new modeling language.
And it is nowhere near Mathlib's scale: a first demonstration.

CogLean's layers

Applications: task models assembled from theories and library parts (multi-digit addition; naming people; learning a device)

Theories: EPAM, strategy choice (DOA), ACT-R declarative memory, concepts, semantic memory, ...

CogLib: representations, memories, processes, decision cycle; a retriever interface; theory-neutral phenomena and laws, with their theorems

Lean 4 (core only): the language and the proof checker; exact rational probabilities

Each layer imports only those below it, and inherits their theorems.

Four kinds of object, kept apart

1. **Mechanisms** and their postulates: definitions (how a net sorts a code; how retrieval samples).
2. **Phenomena**: predicates, stated without any mechanism. A predicate asserts nothing; it is a target.
3. **Theorems**: a mechanism produces behavior satisfying a predicate.
4. **Conditions**: situational ("x is confused with q"), empirical premises not yet explained ("associations are sharp"), well-formedness.

A theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction. The catalog flags these mechanically.

Scientific notions, and what they are in Lean

Model (program)	executable definitions: the same ones that run
Mechanism	a CogLib process, chosen by a theory
Phenomenon	a predicate, with no mechanism: a target
Psychological fact	a phenomenon judged to hold of people: *outside Lean*
Prediction	a theorem's conclusion, not yet observed
Condition	a theorem's hypothesis
Law	a property of a mechanism: proved for a family, or required by a class theorem
Explanation	a theorem and its lemmas
Deduction	what Lean's kernel checks
Illustration	a value computed at one setting: a witness, not an explanation
Lean's axioms	logic only, never scientific assumptions

Meet Chris (and Robin)

- **Chris**: a hypothetical cognitive scientist who studies how children learn arithmetic, and has built models before.
- The usual workflow: write a program that implements the mechanisms; run it; explain the results in a paper.
  - The program specifies the mechanisms.
  - The *reasoning* about their consequences lives in Chris's head, the comments, the prose.
- CogLean supports **programming** a mechanism *and* **reasoning formally** about it; the reasoning is checked, kept as theorems, reused.
- **Robin**: Chris's chatbot research colleague. Good at brainstorming, finding library results, suggesting proofs. *Not* automatically right.

Chris poses the questions · Robin suggests · Lean checks · evidence decides

Chris's two questions

1. **Forward: from an observation to a theorem.**
   People sometimes confidently give the wrong answer, or the wrong name. Does that follow from existing mechanisms, and under what conditions?
   Formalization; a phenomenon stated once; results for classes of memories; more than one explanation.

2. **Backward: what must a theory be like?**
   Chris conjectures a regularity about practice and backup use. Which readings of the theory guarantee it, and which break it?
   Discovery: an exposed ambiguity, a counterexample, a proof, and the assumption the regularity needs.

Then: what if the theory, or the phenomenon, isn't exact? And what all this does and doesn't show.

Part 1

From an observation to a theorem

Does confident error follow from mechanisms we already have?

Chris's observation

- People sometimes **confidently** give a wrong answer, apparently retrieving it rather than working it out.
- A child who knows $100 + 100 = 200$ takes $150 + 100$ for the familiar problem, and says **200**, without hesitating.
- The same shape, with no arithmetic: you greet a stranger, Carl, confidently, as **Bob**.
- Chris's question: does this follow from recognizable properties of *existing* mechanisms?

Robin: CogLib has a general mechanism for exactly this shape,

recognize a stimulus → retrieve an associated response → back up if retrieval fails.

The smallest instance: naming people

- A face is a feature code. Memory recognizes it as a familiar face, or as nothing.
- The recognized person's **name associations** are searched, as in Siegler & Shrager's model: sample a name by strength, state it if it clears a criterion.
- Backup: "don't know".

EPAM (Feigenbaum & Simon): a discrimination net learns Ann's and Bob's faces. They differ in the first feature, so that's all it tests.

Carl shares Bob's first feature, so the net sorts him to Bob.

Own name .90, others .01, criterion .50.

The phenomenon, stated without a mechanism, and when it follows

If q is recognized as itself, x is recognized as q, q's associations are **sharp**, and their correct answers differ:

(i) retrieval never errs on q;

(ii) every retrieved answer to x is wrong;

(iii) x falls back on the backup *exactly as often* as q.

- The pattern is defined once in CogLib, with no mechanism in it: a **target**.
- **Face naming**: Carl is called Bob with probability 45/46, and says "don't know" 1/46 of the time, exactly as often as Bob. For every search length.
- **Conditions**: the confusion (here, a learned net); sharpness (assumed, not yet explained); positive strengths.

In words: the theorem on the next slide

CogLib's `confident_errors`, for *any* task, memory and backup:

- **Parameters**: how problems are coded, what memory calls up (`retrieve`), the associations (`str`), the criterion and search length, and the backup.
- **Hypotheses**:
  - strengths have positive totals; the backup is a proper distribution;
  - q calls up itself, for certain (`hq`);
  - x calls up q, for certain (`hx`): the confusion;
  - q's associations are sharp (`hs`); the correct answers differ (`hne`).
- **Conclusion**: the *confusion-error pattern*, a phenomenon defined once in CogLib, apart from any mechanism: three equations, (i)–(iii) of the last slide.
- **Proof**: three lines, citing three earlier lemmas.

The theorem, in Lean

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (recognizeRetrieve code retrieve str as cc n backup)
      (wholeWrong correct) isWhole isBackup x q := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup
      correct q q (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup
      correct x _ _ (hT _) hx hs hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]

def ConfusionErrorPattern (trial : P → Dist O) (wholeWrong : P → O → Bool)
    (isWhole isBackup : O → Bool) (x q : P) : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧
    prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧
    prob (trial x) isBackup = prob (trial q) isBackup
```

Verbatim from CogLib (`Process/RecognizeRetrieve`, `Phenomena/ConfusionErrors`). Face naming and the multi-digit model are one-line instances.

Equal is not low

- The theorem gives *equal* backup use for x and q, not *low* backup use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the stranger falls back with probability at most $(1 - r)^n$: **confident** errors, a second, quantitative phenomenon.
- With flat associations on q, the same theorem holds, and the errors are not confident.

In face naming, $R_q = 45/46$ and $n = 1$: "don't know" $1/46$ of the time.

Does it need EPAM? Laws, and a rival memory

- The theorem holds for **any memory** with three laws: *determinacy* (one item, for certain), *certainty* (a studied item retrieves itself), *stability* (study doesn't change others).
- The theory matrix proves, for each memory family, which laws it has at every setting:
  - EPAM's net: all three. Face naming follows again, from the laws.
  - ACT-R's declarative memory: **none**, at any setting.

	EPAM	ACT-R
Carl called Bob	45/46	225/391
Carl called Ann	0	225/782
"don't know": Carl / Bob	1/46 / 1/46	107/782 / 229/2254

Not covered is not the same as can't: ACT-R misnames too, gradedly; part (iii) fails.

A second explanation of the same phenomenon

- A **graded** version needs no certainty: if x calls up q, and q itself, each with probability $\geq 1 - \delta$, the pattern holds to within 2δ.
- ACT-R meets it when q is **primed**: for every $\delta > 0$, enough priming suffices. Having *just seen* q is priming, by ACT-R's base level.

	EPAM	primed ACT-R
mechanism	discrimination	partial matching + priming
errors follow	**shared features**	**the context** (who was just seen)

Just after seeing Bob 10 times: Carl called Bob .92; and **Ann** called Bob .80. EPAM: Ann never.

One phenomenon, two explanations, and an observation that tells them apart.

What Chris has now

- Not a simulation of one error, but a **reusable result**: sufficient conditions for the phenomenon, for every setting that meets them.
- It applies to a new task **without re-running** anything (names here; sums in the companion talk), and to *any* memory with three laws.
- The conditions are where the science is:
  - Lean hasn't shown that people have sharp associations, or that a given learning history produces the confusion: separate theoretical or empirical questions;
  - the pattern by itself doesn't make backup use rare; *confident* needs a further, quantitative condition.

Part 2

Working backward

What must a theory be like to guarantee a regularity?

Chris's conjecture

- In DOA, practice strengthens problem–answer associations; retrieval should succeed more often, and overt strategies fade.
- Chris proposes something stronger:

  Once the correct answer is the strongest association, strengthening it further never raises the probability of resorting to a backup strategy.

- Plausible, but an empirical claim about what *never* happens: hard to establish directly.
- **Robin**: first ask whether it follows from DOA.

Chris already knows that formalization exposed an ambiguity in one of the chapter's claims (next slide).

Formalization alone exposed an ambiguity in a theory's claim

The DOA chapter: "a peaked distribution results in less frequent use of overt strategies."

- **Follows**: moving strength to the correct answer never raises retrieval errors, at any setting.
- **Does not follow, for arbitrary redistributions**: it can *raise* overt strategy use.
  - $1 + 1$, strengths $.38, .31, .31 \rightarrow .39, .30, .31$;
  - averaged over the chapter's twelve criteria: $7/12 \rightarrow 73/120$.
  - The weakened wrong answer drops to a criterion, stops being stated, and the child turns to fingers.
  - This *moves* strength; DOA's learning rule only *adds* it.

This disproves the unrestricted claim, not that DOA's *learning* raises backup use. For learning, the answer depends on an assumption the chapter leaves open (next slides).

Robin spots an ambiguity

An answer is drawn in proportion to its strength, and stated only if it **exceeds a criterion** c. Which strength?

Raw: the answer's own strength.

DOA's learning postulate (A6) just *adds* strength; CogLean's `strengthen` does the same.

Normalized: its strength divided by the total.

The chapter *does* renormalize, after finger elaboration (A4); CogLean's `elaborate` follows it.

So "learn, then renormalize" is a natural alternative reading, not a strawman. Do the two readings agree about Chris's conjecture?

The same learning event, two readings

Strengths .38 (correct), .31, .31: summing to one, the correct answer strongest.
Criterion $c = .30$, one attempt.
Every answer exceeds c, so a backup is **never** used. Now learn from the correct answer: $+.05$.

	strengths after learning	backup use
before	$(.38, .31, .31)$	0
raw	$(.43, .31, .31)$	0
renormalized	$\approx (.410, .295, .295)$	$62/105 \approx .59$

Renormalized, the wrong answers fall below .30: still drawn, but rejected. **Opposite consequences** for backup use.

Checked exactly in Lean: `normalized_strengthening_can_raise_backup`.

Robin overreaches

Robin: "So with raw strengths, strengthening the correct answer can *never* raise backup use."

- Too broad. If the correct answer is **below** the criterion and another is above, a small boost makes the acceptable answer *less* likely to be drawn, without yet making the correct one acceptable.
- Lean confirms a case: strengths .2 (correct) and .5, criterion .30, one attempt; add .05 to the correct answer:

$$\text{backup use } \ 2/7 \ \to \ 1/3$$

(`strengthening_below_criterion_can_raise_backup`)

So the condition in Chris's conjecture is doing real work.

The proof, in words

What the proof needs is slightly weaker than dominance: the correct answer is **above the criterion**, or **no answer is**.

- If it's above: strengthening it adds the same amount to the strength *above* the criterion and to the *total*, so their ratio R (one attempt succeeds) can't fall; exhaustion, $(1 - R)^n$, can't rise.
- If none is above: backup use is already certain.
- Dominance implies the condition: if anything is above, the strongest answer is.

Proved for every finite set of answers, nonnegative strengths, criterion and search length:

- CogLib, any memory of this kind: `reach_boost`, `exhausted_boost`;
- DOA's learning rule: `correct_strengthening_never_raises_backup`, `dominant_strengthening_never_raises_backup` (next slide).

The theorem, in Lean

```
theorem dominant_strengthening_never_raises_backup
    (as : List Answer) (s : Answer → Rat)
    (inc cc : Rat) (p : Problem) (hs : ∀ a, 0 ≤ s a) (hinc : 0 ≤ inc)
    (hT : 0 < total as s)
    (hdom : ∀ a ∈ as, s a ≤ s p.sum) (n k : Nat) :
    prob (search as (strengthen s inc p.sum p.sum) cc k n) Retrieval.isExhausted ≤
      prob (search as s cc k n) Retrieval.isExhausted
```

- `hdom`: Chris's condition, the correct answer is the strongest.
- `strengthen`: DOA's own learning rule (A6), no renormalizing.
- Conclusion: exhaustion, so backup use, is no more likely after learning, at every criterion `cc` and search length `n`.
- The proof cites `correct_strengthening_never_raises_backup`, which cites CogLib's `exhausted_boost`.

Arbitrary states versus states learning can reach

- The peaking counterexample *moves* strength from a wrong answer to the correct one.
- DOA's learning rule *adds* strength to the stated answer.
- A counterexample built from an arbitrary redistribution doesn't show the same change can happen **by learning**.
- The renormalized counterexample does arise by learning, but only under the *alternative* rule.

A theorem about every strength distribution answers one question; a theorem about the distributions a learning process can reach answers another. Both are useful. They shouldn't be confused.

What has Chris learned?

- The conjecture has become a way to **tell theoretical commitments apart**.
- If backup use really never rises as a dominant correct answer grows, then renormalized DOA learning, unrestricted, is inadequate.
- Raw strengths are *one* way to satisfy it, not the only one; the renormalized model might be rescued by restricting parameters or reachable states.
- Chris hasn't shown that children use raw strengths. Chris has found which assumptions produce, or fail to guarantee, a proposed regularity.

Chris poses · Robin generates alternatives · formalization exposes the ambiguity · Lean settles what follows · evidence decides

Part 3

When theories and phenomena aren't exact

The power law of practice, unbundled

Imprecise mechanisms

1. **Probabilistic, with known probabilities**: still precise. The outcome distribution is a mathematical object; CogLib computes and reasons about it **exactly**. (Everything so far.)
2. **Monte Carlo**: the distribution is defined, but expensive. Sampling makes the consequences harder to calculate, not the theory imprecise. Results are **estimates**, checked against exact values where both exist.
3. **Incompletely specified**: a heuristic, a table, an unknown process. It becomes a **parameter**, and what it must satisfy becomes a **condition**. (Sharp associations.)

A model can mix deduced and estimated results, if it says which is which.

Imprecise phenomena: the power law, unbundled

"Practice follows a power law" bundles claims of different strength:

improves $\supset$ diminishing returns $\supset$ diminishing *relative* returns

$\supset$ approximately a power law (within ε, over a range) $\supset$ exactly a power law

- Each is a predicate on a curve, in CogLib, with exact rationals: a power law is **scale invariance**, $T(kn) = r\,T(n)$.
- Proved without any mechanism: power and exponential curves are **different** phenomena.
- **Averaging**: the mean of two exponential learners is *not* exponential, and has a power curve's signature. (Estes 1956; Heathcote et al. 2000.)

Whether noisy data "look like" a power law is a statistical question; these are mathematical ones.

One mechanism, two practice schedules

ACT-R's base level: each use leaves a decaying trace; retrieval time falls as activation rises. With decay 1:

- **Massed, tested after a delay**: latency $= FL/n$. An **exact** power law.
- **Spaced, tested after each trial**: improves, diminishing returns, diminishing relative returns: *every* qualitative signature. But **not** a power law over long ranges; it's logarithmic.
  - doubling ratio $T(2n)/T(n)$: $.67, .72, .77, .80, .83, .86, \cdots \to 1$
  - yet over trials 1–8, within 7.5% of a power law with exponent $\approx .47$

With decay $d < 1$ (over the reals): approximately a power law, exponent $1 - d$.

So the power law depends on the **schedule** and the **decay**; and the signature doesn't identify the mechanism.

Part 4

What this shows, and what it doesn't

Kinds of results, where it sits, limits

What each result rests on

Kind	Examples
Library facts	a sample is a probability distribution; two evaluators agree
Compositional (every setting)	after certain recognition, backup use depends only on what's recognized
Conditional	confident errors; ACT-R's practice curves, given a schedule
Class-level	any memory with three laws; any non-interfering learner reaches criterion
Relations between phenomena	power $\neq$ exponential; averaged exponentials aren't exponential
At chosen settings	45/46; 225/391; within 7.5% over trials 1–8

None of these is yet an empirical finding about people. The numbers are witnesses, not explanations, and not fits.

Model verification and theoretical deduction

- **Simulation** samples executions.
- **Probabilistic model checking** (PRISM; Soar to PRISM, Ganeriwala et al. 2026) analyzes a specified state-transition model exhaustively; it is formal verification, and can treat parameters symbolically.
- **Deductive proof** (CogLean) establishes results for whole classes of models satisfying theoretical assumptions.

The contrast is *model verification* versus *theoretical deduction*. Complementary, not competitors.

CogLean's claim is not formal methods (see the prior work), but the combination: a shared, cumulative library with reusable proofs, cross-theory composition, and deductive discovery.

Relationship to cognitive architectures

- **Complementary**, not an alternative to Soar, ACT-R, or the Common Model of Cognition.
- Architectures accumulate shared mechanisms and models; CogLean tries to accumulate **proved results** about mechanisms, including over classes of them. An architecture could carry such results itself.
- Example: confident errors hold for every memory with three laws (EPAM has them; ACT-R's memory has none). The laws are sufficient, not shown necessary; no general equivalence or divergence of architectures is claimed.
- A possible **formal foundation** for expressing, composing and reasoning about their mechanisms and architectural commitments.
- The present library composes locally, for particular tasks, without committing to a general architecture.
- Longer term: formalize **architectural constraints themselves**, so their implications can be proved and architectures compared in one framework.

What a proof shows, and what it doesn't

- Lean guarantees **validity**: conclusions follow from premises.
- Not **empirical adequacy**: whether the premises describe people.
- Not **explanation**: whether the premises already contain the phenomenon.
- Example: a serial-position result whose mechanism was a bowed schedule. Valid, nearly empty. Now derived from an attention rule, which also gives the isolation effect.
- An explicit audit sorts results as definitional, close, derived, compositional.

What CogLean is, and isn't

It doesn't

- generate hypotheses or theories: people (or AIs) propose them;
- find phenomena in data, or fit parameters;
- show that a theory is true of people;
- decide whether a derivation is an explanation;
- replace simulation or experiment.

It does

- check whether a proposed consequence follows, and from exactly what;
- find counterexamples to informal claims (peaking; renormalized learning);
- expose unstated assumptions and conditions;
- say for which class of models a result holds;
- let proved results be reused across theories and tasks.

"Discovery" here means what Chris did: finding out what does, or doesn't, follow from theories people propose.

Directions

- **Composing theories** (companion talk): EPAM + strategy choice + a problem solver → an executable model of three-digit addition, with consequences proved largely from the parts' theorems.
- **A second domain** (companion): concept formation by view application, on the same memory interface and laws.
- **Classes of mechanisms**: more families and laws; graded laws for graded phenomena.
- **Quantitative phenomena**: other theories' learning curves on the same predicates; statistical versions, for noisy data.
- **Several explanations**: what primes; when the explanations diverge.

Chris and Robin: from reasoning to model construction

A future possibility, not a present accomplishment.

Chris: Confident errors follow from three laws of memory. Which memories could I build a new model on?

Robin: Here is what CogLib has proved:

- **EPAM's net:** all three, once your encoding makes the stimuli noticeably different.
- **Global matching:** determinacy only, so far; certainty and stability are open obligations.
- **ACT-R's memory:** never, so this route is closed (other routes aren't).

Chris: So I could assemble a model from mechanisms whose proved properties fit together?

From mechanically reasoning about models
to mechanically assisted model construction.

Limitations

- No contact with data yet; plausibility is for evidence and judgment.
- Key conditions are assumed, not derived (sharp associations); strengths are chosen, not learned.
- Exact rationals: ACT-R mostly at decay 1 and temperature 1 (reals only for learning curves), and without retrieval as use.
- Scalability: exact finite distributions; continuous and neural models need other techniques.
- Nonnegative weights not enforced by the type, nor proved library-wide.
- A small library, of theories chosen partly because they could be formalized.

Take-home

- Theories stated once, in a shared library; phenomena stated apart from mechanisms, at the precision the evidence warrants; results for whole classes that satisfy stated laws.
- A phenomenon can have several explanations, and the library can say what observation tells them apart.
- It works both ways: forward from a phenomenon to its sufficient conditions; backward from a regularity to the assumptions it needs.
- Every result is labelled: library fact, compositional, conditional, class-level, or a witness.
- Theorems are **explanations**, proved about the programs that run, so they stay aligned with the models as both change.

Not formalizing proofs, but formalizing explanations,
so that we can reason about them. The goal: a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and traditions.

`https://github.com/jeffshrager/CogLean`